Fine-Tuning Your LLM

Using large language models to build generative AI applications comes with new challenges. A key issue is ensuring response quality (accuracy and relevance) in content generated by the model for a given user request. In previous lessons, we discussed techniques like prompt engineering and retrieval-augmented generation that try to solve the problem by *modifying the prompt input* to the existing model.

In today's lesson, we discuss a third technique, **fine-tuning**, which tries to address the challenge by *retraining the model itself* with additional data. Let's dive into the details.

Learning Objectives

This lesson introduces the concept of fine-tuning for pre-trained language models, explores the benefits and challenges of this approach, and provides guidance on when and how to use fine tuning to improve the performance of your generative AI models.

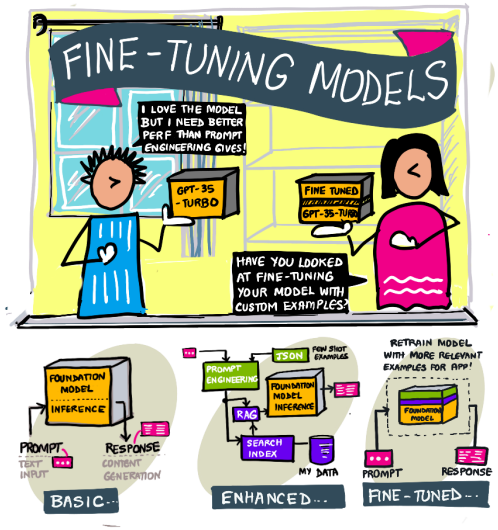
By the end of this lesson, you should be able to answer the following questions:

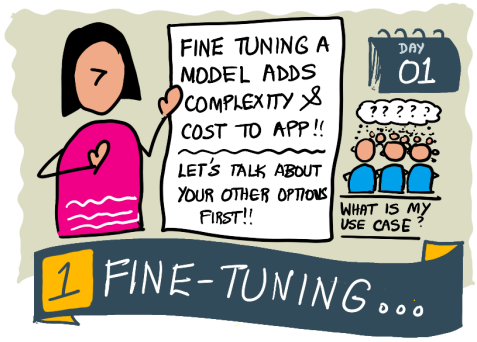
- What is fine tuning for language models?
- When, and why, is fine tuning useful?
- How can I fine-tune a pre-trained model?
- What are the limitations of fine-tuning?

Ready? Let's get started.

Illustrated Guide

Want to get the big picture of what we'll cover before we dive in? Check out this illustrated guide that describes the learning journey for this lesson - from learning the core concepts and motivation for fine-tuning, to understanding the process and best practices for executing the fine-tuning task. This is a fascinating topic for exploration, so don't forget to check out the Resources page for additional links to support your self-guided learning journey!

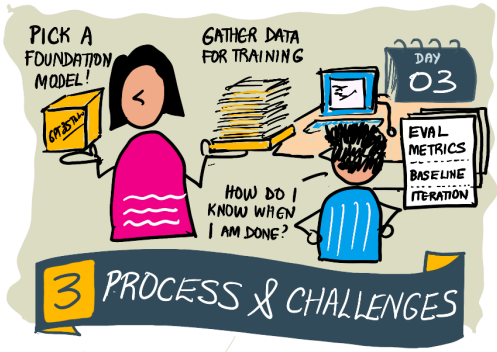




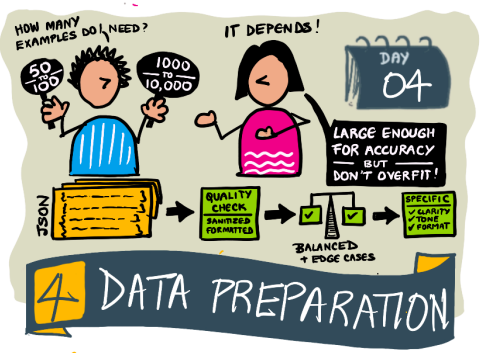
- FOUNDATION MODELS
- PROMPT ENGINEERING
- RETRIEVAL AUGMENTED GENERATION
- MODEL FINE TUNING



- TOKENIZATION COSTS
- PROMPT ENGINEERING LIMITS
- APPLICATION DOMAIN NEEDS
- UPSKILLING CHEAPER MODELS



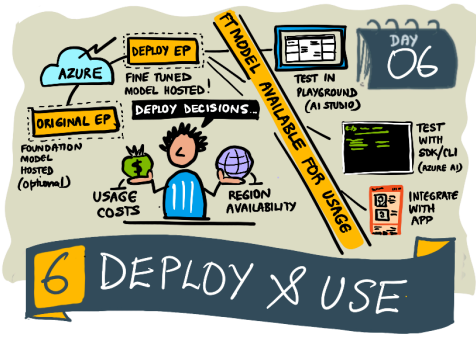
- DATA GATHERING & FORMATTING
- MODEL TRAINING & COMPLEXITY
- MODEL EVALUATION & ITERATION
- MODEL DEPLOYMENT & MAINTENANCE



- SIZE OF TRAINING DATA (#examples)
- FORMAT OF EXAMPLE (model based)
- DATA REPRESENTATION (fairness)
- REQUIREMENTS COVERAGE (quality)



- UPLOAD DATA FOR TRAINING
- CREATE FINE TUNING JOB
- MONITOR JOB STATUS
- TEST FINE TUNED MODEL



- REGION AVAILABILITY (constraint)
- MODEL RATE LIMITS (shared)
- USE IN PLAYGROUND (validate)
- USE IN SDK/APP (integrate)



- HAVE A VALID USE CASE
- TRY OTHER OPTIONS FIRST
- ESTIMATE COSTS & TRADEOFFS
- HAVE A MAINTENANCE STRATEGY

SKETCHNOTES BY NITYA

What is fine-tuning for language models?

By definition, large language models are *pre-trained* on large quantities of text sourced from diverse sources including the internet. As we've learned in previous lessons, we need techniques like *prompt engineering* and *retrieval-augmented generation* to improve the quality of the model's responses to the user's questions ("prompts").

A popular prompt-engineering technique involves giving the model more guidance on what is expected in the response either by providing *instructions* (explicit guidance) or *giving it a few examples* (implicit guidance). This is referred to as *few-shot learning* but it has two limitations:

- Model token limits can restrict the number of examples you can give, and limit the effectiveness.
- Model token costs can make it expensive to add examples to every prompt, and limit flexibility.

Fine-tuning is a common practice in machine learning systems where we take a pre-trained model and retrain it with new data to improve its performance on a specific task. In the context of language models, we can fine-tune the pre-trained model *with a curated set of examples for a given task or application domain* to create a **custom model** that may be more accurate and relevant for that specific task or domain. A side-benefit of fine-tuning is that it can also reduce the number of examples needed for few-shot learning - reducing token usage and related costs.

When and why should we fine-tune models?

In *this* context, when we talk about fine-tuning, we are referring to **supervised** fine-tuning where the retraining is done by **adding new data** that was not part of the original training dataset. This is different from an unsupervised fine-tuning approach where the model is retrained on the original data, but with different hyperparameters.

The key thing to remember is that fine-tuning is an advanced technique that requires a certain level of expertise to get the desired results. If done incorrectly, it may not provide the expected improvements, and may even degrade the performance of the model for your targeted domain.

So, before you learn "how" to fine-tune language models, you need to know "why" you should take this route, and "when" to start the process of fine-tuning. Start by asking yourself these questions:

- **Use Case**: What is your *use case* for fine-tuning? What aspect of the current pre-trained model do you want to improve upon?
- **Alternatives**: Have you tried *other techniques* to achieve the desired outcomes? Use them to create a baseline for comparison.
    - Prompt engineering: Try techniques like few-shot prompting with examples of relevant prompt responses. Evaluate the quality of responses.
    - Retrieval Augmented Generation: Try augmenting prompts with query results retrieved by searching your data. Evaluate the quality of responses.

- **Costs**: Have you identified the costs for fine-tuning?
  - Tunability - is the pre-trained model available for fine-tuning?
  - Effort - for preparing training data, evaluating & refining model.
  - Compute - for running fine-tuning jobs, and deploying fine-tuned model
  - Data - access to sufficient quality examples for fine-tuning impact
- **Benefits**: Have you confirmed the benefits for fine-tuning?
  - Quality - did fine-tuned model outperform the baseline?
  - Cost - does it reduce token usage by simplifying prompts?
  - Extensibility - can you repurpose the base model for new domains?

By answering these questions, you should be able to decide if fine-tuning is the right approach for your use case. Ideally, the approach is valid only if the benefits outweigh the costs. Once you decide to proceed, it's time to think about *how* you can fine tune the pre-trained model.

Want to get more insights on the decision-making process? Watch To fine-tune or not to fine-tune

How can we fine-tune a pre-trained model?

To fine-tune a pre-trained model, you need to have:

- a pre-trained model to fine-tune
- a dataset to use for fine-tuning
- a training environment to run the fine-tuning job
- a hosting environment to deploy fine-tuned model

Fine-Tuning on Microsoft Foundry

Microsoft Foundry is where you fine-tune, deploy, and manage custom models on Azure today (it brings together what used to be Azure OpenAI Studio and Azure AI Studio). Before you start a job, it helps to understand the choices Foundry gives you - and the best practices the platform recommends. Under the hood, Foundry uses **LoRA (low-rank adaptation)** to fine-tune models efficiently, which keeps training faster and more affordable than retraining every weight.

Step 1: Pick a training technique

Foundry supports three fine-tuning techniques. **Start with SFT** - it covers the widest range of scenarios.

Technique	What it does	When to use it
Supervised Fine-Tuning (SFT)	Trains on input/output example pairs so the model learns to produce the responses you want.	The default for most tasks: domain specialization, task performance, style and tone, instruction-following, and language adaptation.
Direct Preference Optimization (DPO)	Learns from *preferred vs. non-preferred* response pairs to align outputs with human preferences.	Improving response quality, safety, and alignment when you have comparative feedback.

Technique	What it does	When to use it
Reinforcement Fine-Tuning (RFT)	Uses reward signals from *graders* to optimize complex behaviors with reinforcement learning.	Objective, reasoning-heavy domains (math, chemistry, physics) with clear right/wrong answers. Requires more ML expertise.

Step 2: Pick a training tier

Foundry lets you choose how and where training runs:

- **Standard** - trains in your resource's region and guarantees data residency. Use it when data must stay in a specific region.
- **Global** - cheaper and faster to queue by using capacity beyond your region (data and weights are copied to the training region). A good default when data residency isn't a requirement.
- **Developer** - the lowest cost, using idle capacity with no latency/SLA guarantees (jobs can be preempted and resumed). Ideal for experimentation.

Step 3: Choose a base model

Fine-tunable models include OpenAI `gpt-4o-mini`, `gpt-4o`, `gpt-4.1`, `gpt-4.1-mini`, and `gpt-4.1-nano` (SFT; the 4o/4.1 family also supports DPO), the reasoning models `o4-mini` and `gpt-5` (RFT), plus open-source models such as `Ministral-3B`, `Qwen-32B`, `Llama-3.3-70B-Instruct`, and `gpt-oss-20b` (SFT on Foundry resources). Always check the current Fine-tuning models list for supported methods, regions, and availability.

> Foundry offers two modalities: **serverless** (consumption-based pricing, no GPU quota to manage, OpenAI and selected models) and **managed compute** (bring-your-own VMs via Azure Machine Learning for the widest model range). Most people should start with serverless.

Foundry best practices

- **Baseline first.** Measure the base model with prompt engineering and RAG *before* you fine-tune, so you can prove the gain.
- **Start small, then scale.** Begin with 50-100 high-quality examples to validate the approach, then grow to 500+ for production. Quality beats quantity - prune low-quality examples.
- **Format data correctly.** Training and validation files must be JSONL, UTF-8 **with a BOM**, under 512 MB, using the chat-completions message format. Always include a validation file so you can watch for overfitting.

- **Keep the training system prompt at inference.** Use the same system message when you call the model that you used during training.
- **Evaluate checkpoints - don't blindly deploy the last one.** Foundry keeps the last three epochs as deployable checkpoints; pick the one that generalizes best by watching `train_loss` / `valid_loss` and token accuracy.
- **Measure token cost alongside quality** when comparing the fine-tuned model to the baseline.
- **Iterate with continuous fine-tuning.** You can fine-tune an already fine-tuned model on new data (supported for OpenAI models).
- **Mind hosting costs.** A deployed custom model bills hourly, and an inactive deployment is removed after 15 days - clean up what you don't need.

Work through the end-to-end walkthrough in Customize a model with fine-tuning, and see the guides for DPO and RFT when you're ready for the other techniques.